NEXUS

E-Commerce Analytics Platform

Technical Analysis Report

CSE 214 — Advanced Application Development | Spring 2026

Course	CSE 214 — Advanced Application Development
Members	Elif Buse Çınar, Musa Talat Demir
GitHub	github.com/Talatd/e-commerce-project
University	Akdeniz University, Antalya, Turkey
Date	April 2026

1. Project Overview

Nexus is an e-commerce data analytics platform serving three user roles — Admin, Corporate, and Individual — with role-appropriate dashboards, full CRUD functionality, and an AI-powered Multi-Agent Text2SQL chatbot. The platform integrates six Kaggle datasets into a unified relational schema and is built on Spring Boot, Angular, and a Python/LangGraph AI Hub.

Role	Key Responsibilities
Admin	User management, store approval/closure, platform analytics, audit logs, global config
Corporate User	Store dashboard, inventory, sales analytics, order fulfilment, customer segmentation
Individual User	Browse products, shopping cart, order tracking, purchase history, reviews, personal analytics

2. System Architecture

The platform uses a decoupled four-layer architecture. Each layer communicates over defined interfaces: REST/JSON for synchronous operations, WebSocket (STOMP) for real-time events, and an internal HTTP channel between Spring Boot and the Python AI Hub.

Layer	Technology	Role
Presentation	Angular 17+, Chart.js, D3.js	Responsive dashboards, real-time WebSocket updates, chat UI
Application	Spring Boot 3.x (Java 17)	REST API, JWT auth, Spring Security RBAC, Swagger docs
AI Hub	Python, LangGraph, OpenAI / Claude	NL → SQL pipeline, 5-agent orchestration, Plotly visualisation
Data	MySQL / PostgreSQL, JPA/Hibernate	Normalised schema, indexes, ETL-seeded Kaggle data

Architecture Decisions

- Spring Boot chosen for its unified Security + JPA + WebSocket ecosystem; dual MySQL/PostgreSQL support via a single property change.
- Angular 17+ for TypeScript safety, lazy-loaded feature modules, and first-class RxJS/WebSocket integration.
- LangGraph models the chatbot as an explicit state machine DAG — control flow is auditable, nodes are pure functions, and the LLM provider is swappable.
- Decoupled Python AI Hub: isolates LLM credentials, allows independent scaling, and avoids JVM-based LLM workarounds.

3. Database Design & ER Diagram

The schema integrates six Kaggle source datasets, normalised to Third Normal Form (3NF). All tables use surrogate BIGINT primary keys; foreign key constraints are enforced at the database level.

3.1 Schema

```
USERS          (id, email, password_hash, role_type, gender)
  ├─ STORES            (id, owner_id FK, name, status)
  ├─ CUSTOMER_PROFILES(id, user_id FK, age, city, membership_type)
  └─ ORDERS            (id, user_id FK, store_id FK, status, grand_total)
       ├─ ORDER_ITEMS  (id, order_id FK, product_id FK, quantity, price)
       └─ SHIPMENTS    (id, order_id FK, warehouse, mode, status)

PRODUCTS (id, store_id FK, category_id FK, sku, name, unit_price)
  └─ REVIEWS (id, user_id FK, product_id FK, star_rating, sentiment_score)

CATEGORIES (id, name, parent_id FK)   -- self-referencing hierarchy
```

3.2 Entity Relationship Diagram

Figure 1 — Nexus Platform ER Diagram (3NF · FK constraints enforced at DB level)

3.3 Dataset → Table Mapping

Target Table	DS1 Online Retail	DS2 Behavior	DS3–DS6 Other
USERS	CustomerID	Gender, Age, City, Membership	DS6: CustomerID
PRODUCTS	StockCode, Desc, UnitPrice	—	DS4: SKU, Category; DS5: SKU; DS6: Title
ORDERS	InvoiceNo, Date	—	DS4: Status, Fulfilment; DS5: GrandTotal, PaymentMethod

Target Table	DS1 Online Retail	DS2 Behavior	DS3–DS6 Other
SHIPMENTS	—	—	DS3: Warehouse, Mode, Discount; DS4: ServiceLevel
REVIEWS	—	SatisfactionLevel	DS6: StarRating, HelpfulVotes, TotalVotes
CUSTOMER_PROFILES	—	TotalSpend, ItemsPurchased, DiscountApplied	—

4. ETL Process Documentation

A custom DbInitializer bean manages the full Extract → Transform → Load lifecycle. In demo mode it executes a "Clean Sweep" on startup: truncate all tables, then reload from JSON master files — guaranteeing consistent state for every demo session.

4.1 Pipeline

Stage	Input	Output	Key Steps
Extract	6 Kaggle CSV files	Master JSON files	CSV parsed with OpenCSV; one JSON per entity
Transform	Master JSON	Cleansed records	Field mapping, de-dup, date normalisation, currency conversion
Load	Cleansed records	Populated DB	Transactional batch inserts; FK resolution order enforced

4.2 Transformation Rules

Field Mapping & Cleansing

- Null CustomerID → synthetic ID from hash(InvoiceNo + rowIndex)
- Zero/negative UnitPrice → replaced with category median
- Duplicate InvoiceNo+StockCode → keep row with latest InvoiceDate
- All dates → ISO 8601 UTC via Java DateTimeFormatter
- Non-UTF-8 characters in descriptions → normalised with Apache Commons Text

Financial Transformations

- Supplier price = UnitPrice × 0.65 (DS1)
- DS5 PKR amounts → USD at 0.0036 (stored in exchange_rate_metadata table)
- GST (18%) and shipping cost stored as separate ORDERS columns

Synthetic Enrichment

- sentiment_score (0.0–1.0): derived from DS2 SatisfactionLevel and DS6 StarRating
- Regional stock (Global / Europe / Asia): generated from DS3 CostOfProduct ranges
- Order event log (PLACED → CONFIRMED → SHIPPED → DELIVERED): seeded for 50+ orders

5. Backend Implementation (Spring Boot)

Spring Boot 3.x (Java 17) — RESTful API, JWT authentication with refresh token rotation, Spring Security RBAC, JPA/Hibernate for both MySQL and PostgreSQL, Swagger/OpenAPI 3.0 documentation, and a dedicated /api/chat/ask endpoint for safe AI-generated SQL execution.

5.1 Key Endpoints

Endpoint	Method(s)	Access
/api/auth/login, /refresh, /logout	POST	Public
/api/products	GET / POST / PUT / DELETE	GET: all roles | mutations: Corporate, Admin
/api/orders	GET / POST / PATCH	Individual (own), Corporate (store), Admin (all)
/api/analytics/**	GET	Role-filtered KPIs, trends, segmentation
/api/chat/ask	POST	All authenticated roles (data scoped per role)
/api/admin/users, /stores	GET / PATCH / DELETE	Admin only
/ws (WebSocket STOMP)	—	All authenticated roles

5.2 Security

- JWT: 15-min access tokens + 7-day refresh tokens in HttpOnly cookies
- @PreAuthorize on every service method — role checks are co-located with business logic
- Chatbot SQL validated via table/column whitelist before execution; read-only DB user for chatbot connection
- Global @ControllerAdvice returns structured JSON error envelopes for all exceptions

6. Frontend Implementation (Angular)

Angular 17+ with TypeScript, mobile-first responsive layout, lazy-loaded feature modules, and service-based state management. Real-time KPI cards subscribe to WebSocket topics via RxJS observables.

6.1 Module Structure

Module	Contents
CoreModule	JWT interceptor, auth guards, global error handler
DashboardModule	Role-adaptive KPI widgets, revenue/trend charts, activity feed
ProductsModule	Catalogue grid, product detail, inventory management (corporate)
OrdersModule	Checkout, order timeline, fulfilment workflow
AnalyticsModule	Configurable chart widgets, date-range pickers, export
ChatModule	AI assistant UI — streaming responses, inline Plotly charts
AdminModule	User management, store approvals, audit log (admin only)

6.2 Visualisation

- Chart.js — revenue trend lines, order status doughnuts, KPI sparklines
- D3.js — customer geographic heatmap, category hierarchy treemap
- Plotly — dynamic charts generated by the AI Visualisation Agent, rendered inline in chat bubbles

7. Multi-Agent Text2SQL AI Chatbot

The chatbot converts plain English questions into SQL, executes them safely, explains the results, and optionally renders a Plotly chart — without requiring any SQL knowledge from the user. It is orchestrated by LangGraph as an explicit state machine.

7.1 Agents

Agent	Role	Responsibility
Guardrails Agent	Scope Manager	Classifies input: GREETING → reply & end | OUT_OF_SCOPE → reject | IN_SCOPE → continue
SQL Agent	SQL Expert	Generates valid SQL from NL using full schema context; no markdown, no explanation
Error Agent	Error Recovery	Diagnoses failed SQL with schema context; rewrites and retries (max 3×)
Analysis Agent	Data Analyst	Translates result rows into plain-English business insights with specific numbers
Visualisation Agent	Plotly Expert	Generates executable Plotly code when a chart adds value; skipped otherwise

7.2 Agentic Flow

User → Guardrails → SQL Agent → Execute SQL → (Error Agent on fail, max 3 retries) → Analysis Agent → Decide Graph Need → (Visualisation Agent if needed) → Response to Angular UI

7.3 State (TypedDict)

```
class AgentState(TypedDict):
    question, sql_query, query_result, error,
    final_answer, visualization_code, is_in_scope, iteration_count
```

7.4 Role-Scoped Access

Role	Chatbot Data Scope
Individual	Own orders, purchases, reviews, and spending patterns only
Corporate	Own store's products, orders, customers, and sales data
Admin	Full platform access — all stores, users, aggregate metrics

7.5 Security

- Generated SQL validated against table/column whitelist before execution

- Read-only DB user (nexus_chatbot) — SELECT only, DDL/DML blocked at DB level
- Role-scoped CTE injected as prefix: WITH scoped_data AS (...) wraps every chatbot query
- 10-second query timeout prevents long-running analytical queries from affecting OLTP load

8. Role-Based Access Control

RBAC is enforced at three independent layers: URL-level SecurityFilterChain rules, method-level @PreAuthorize annotations, and data-layer WHERE clauses that bind queries to the authenticated user's permitted scope.

Feature	Admin	Corporate	Individual
Platform analytics & audit logs	✓	—	—
User management (suspend/delete)	✓	—	—
Store approval & management	✓	✓	—
Inventory & product CRUD	✓	✓	—
Sales analytics & revenue reports	✓	✓	—
Order placement & tracking	—	—	✓
Product browsing & reviews	—	✓	✓
Personal spending analytics	—	—	✓
AI chatbot (role-scoped)	✓	✓	✓

9. API Documentation Overview

Full Swagger UI is available at /swagger-ui.html; the static OpenAPI 3.0 spec is committed at /docs/api-spec.json. All endpoints follow REST conventions: noun-based URLs, HTTP verbs for operations, consistent JSON envelopes.

Response Envelope

```
// Success:  { "success": true,  "data": {...}, "meta": { "page":1, "total":248 } }
// Error:    { "success": false, "error": { "code": "FORBIDDEN", "message": "..." } }
```

Chat Endpoint

```
POST /api/chat/ask   (Authorization: Bearer <token>)
Request:  { "question": "Show top 5 products by revenue this month" }
Response: { "answer": "...", "sql": "SELECT ...", "rows": [...], "plotlyCode": "..." }
```

10. Challenges & Solutions

Challenge	Solution
6 Kaggle datasets with conflicting CustomerID formats	Hash-based surrogate key generation; natural keys preserved for auditing
SQL injection via LLM-generated queries	Whitelist validator + read-only DB user + role-scoped CTE prefix
LLM hallucinating invalid column names	Full schema DDL injected into SQL Agent prompt; Error Agent retries with corrected hints
Currency inconsistency (GBP, USD, PKR)	exchange_rate_metadata table; ETL normalises all amounts to USD at load time
Demo data inconsistency between team members	Clean-sweep ETL on startup from idempotent JSON master files
Real-time dashboard without constant polling	WebSocket STOMP broker; Angular components subscribe to /topic/ channels via RxJS

11. Technology Stack Summary

Component	Technology	Notes
Backend	Spring Boot 3.x, Java 17	REST, JWT, RBAC, JPA/Hibernate
Frontend	Angular 17+, TypeScript	Lazy modules, RxJS, WebSocket
Database	MySQL / PostgreSQL	Switchable via application.properties
AI Orchestration	LangGraph (Python)	State-machine multi-agent DAG
LLM	OpenAI gpt-4o-mini / Anthropic Claude	Configurable via .env
Visualisation	Chart.js, D3.js, Plotly	Static + AI-generated charts
Real-time	WebSocket (STOMP/SockJS)	Live stock & order updates
API Docs	Swagger / OpenAPI 3.0	/swagger-ui.html
ETL Engine	Java DbInitializer + Jackson JSON	Clean-sweep idempotent seeding
Bonus: Containers	Docker + Docker Compose	Full-stack local orchestration
Bonus: CI/CD	GitHub Actions	Build, test, Docker push on PR merge

12. Repository

GitHub: github.com/Talatd/e-commerce-project
Branching: main (production-ready) · develop (integration) · feature/* · hotfix/*

```
ecommerce-project/
├── backend/          Spring Boot — src/, resources/etl/, docs/api-spec.json
├── frontend/         Angular — src/app/modules/
├── ai-hub/           Python LangGraph — agents/, graph.py, requirements.txt
├── docker-compose.yml
└── README.md
```